

SCHOOL OF
OPERATIONS RESEARCH AND INFORMATION ENGINEERING
COLLEGE OF ENGINEERING
CORNELL UNIVERSITY
ITHACA, NEW YORK 14853

Financial Engineering Project

HIGH FREQUENCY TRADING STRATEGY FOR 30 YEAR TREASURY BOND

Presented to the Faculty of the Graduate School of Cornell University
in partial fulfillment of the
requirements for the Master of Engineering Degree
and the Financial Engineering Concentration

By:

Team members: Shuo Shen, Jiaxin Yin, Yikai Hua, Tengxiao Li,
Zhengyi Fang, Yunze Huang, Colin Basco

Sponsor: Peter Decrem

Advisor: Sasha Stoikov

December 2019

Faculty Advisor's Signature

2019/12/14
Date

Abstract

We construct new features based on order book data and separate them into three groups, e.g., time-insensitive features, time-sensitive features and cointegration features. For time-insensitive features, we applied serval transformation on imbalance in different levels, and some other features based on order book data. For time-sensitive features, we constructed features with historic information. Besides, we extracted information about deleting and adding order book to construct features on it. For cointegration, we applied linear regression, online regression and Kalman filter to both the treasury data and the corresponding futures data to construct cash and futures cointegration features separately. Then, we predicted the fair-price for each quote given each single feature and combination of features. At last, we designed two smart algorithms to trade 30 Year Treasury Bond given the predicted fair-price. We found that combination features from different groups can help to reduce transaction cost by 95% compared with one tick-size. We believe that the new features we constructed can extract more information from order book, and can be very effective for trading strategies.

Key words: 30 Year Treasury Bond, Online Regression, Cointegration, Kalman Filter

1. Introduction

Existing trading and “mean-reversion” strategies that Citi implements for short-end US Treasuries tend not to work for long-end, 30-year US Treasuries. This paper seeks to determine features and models to understand behavior in the long end of the yield curve. From these features and models, a smart buy and sell algorithm is created that reduces transaction costs of trades by over 90% of a tick size.

Two smart algorithms test the performance of various feature combinations. Both algorithms have a buy only component and a sell only component. The first smart algorithm is the 30-Second Sampling Strategy. Every 30 seconds, the algorithm decides whether to buy/sell, or wait. The algo trades the first sample that triggers a signal within 5-minute intervals. If the algorithm fails to buy/sell in the first nine 30 second intervals, then it must buy/sell in the last 30 second interval. The second algorithm is the Tick-by-tick Strategy. This algo trades the first sample that triggers a buy/sell signal within five-minute intervals. The strategy can trade only once per 5-minute intervals and must buy/sell the last sample if it hasn’t traded yet in a 5-minute interval.

Both algorithms make trading decisions based on predictions of the VWAP 6 million. The VWAP, volume weighted average price, reflects the average price of a security traded throughout a volume-based period. Since the VWAP indicates both the trend and value of a security, it is used in conjunction with the Mid-price as the response variable. Before the VWAP can be calculated, data must be collected and pre-processed.

2. Data Pre-processing

Data is extracted between 10:30 PM on July 1st, 2019 and 5:00 PM on July 2nd, 2019. Three separate files are created for analysis: 1) Levels Data, 2) Trade Data, and 3) Order Book data. The Levels file reflects a snapshot of the supply and demand on the top five bid and ask prices of 2, 3, 5, 7, 10, and 30 Year US Treasury Bonds. The different levels record both the bid/ask price and quantity. Data for the 2, 3, 5, 7, and 10 Year bonds are used in Cash Cointegration and Spot Future Cointegration. The Trade file reflects the

price and quantity for each trade of the 5 Year, 10 Year, and 30 Year US Treasury Bonds. The 5-and-10-year data is used to create a benchmark to measure the performance of Cointegration features. Finally, the Order Book file contains addition and deletion information for the 30-year UST. Both Order Book and Trade files are reconstructed from raw Citi data.

For each trading record j within the Trade data, the VWAP 6 million is calculated accordingly:

$$VWAP_{6j} = \frac{\sum P_i * TS_i}{\sum TS_i},$$

TS_i = Trade size of i order
 P_i = Price of i trade
 $\sum TS_i$ = 6 million
 $\text{Timestamp}_i \geq \text{Timestamp}_j$

Finally, the Order Book and Trades files are merged using the as-of-join in Python for feature construction.

3. Features Construction

3.1 Time-insensitive Features

3.1.1 Symmetrized Imbalance & Transformations

Symmetrized Imbalance, defined as:

$$\text{Imbalance} - 0.5 = \frac{Q_B}{Q_B + Q_A} - 0.5, \quad \begin{matrix} Q_B = \text{Bid Quantity} \\ Q_A = \text{Ask Quantity} \end{matrix}$$

may help predict the VWAP. Generally, a higher imbalance with an excess of ‘bid’ contracts than ‘ask’ contracts increases the probability for upward UST price movement. Imbalances for all five levels of ask and bid prices were calculated separately. For example, “imbalance level x ” calculates the imbalance between the x best bid and x best ask quantities.

Five transformations of imbalance are tested to improve the fit of predictive models. These transformations include square, cube, z-score, log, and tan. A square transformation reduces the left skewness of data. For negative values of symmetrized imbalance, the negative of the square is taken. A cube transformation incorporates higher order fit and also reduces left skewness. A z-score transformation re-scales and normalizes the data. The tan transformation offers another non-linear transformation to improve fit.

3.1.2 Cumulative Volume Difference

The cumulative volume difference, defined as:

$$\text{Sumqdiff} = \sum_{i=1}^5 (V_i^{\text{ask}} - V_i^{\text{bid}})$$

may help predict the VWAP. We construct this feature based on five levels of ask and bid volume in order book data.

We find that the cumulative volume difference is negatively correlated with VWAP, which matches our intuition. This feature is like a general version of imbalance. For example, if more and more investors want to sell the bonds, ask volume of each level increasingly larger than bid volume. Then, the future VWAP would probably decrease due to strong selling power.

3.2 Time-sensitive Features

3.2.1 Last Mid-price Change

We transform the last mid-price change into a dummy variable using the following formula. When the last mid-price change is positive, the dummy variable would be 1; otherwise, it would be -1.

$$\text{dummy_mid_change} = \begin{cases} 1, & \text{if last mid price change} > 0 \\ -1, & \text{if last mid price change} < 0 \end{cases}$$

To construct this feature, we only record the change of mid-price in each tick of order book data. To be specific, for the current tick, if the mid-price does not change, we would not record it. We used the last change of mid-price among earliest ticks as our feature.

We find that the last mid-price change has a negative relationship with VWAP. It indicates that there is a mean-reverting effect in price movement of 30-year Treasury bonds.

3.2.2 Delta Cross-Level Ask/Bid Difference

In order book data, we have five levels of ask and bid volume. For two adjacent levels, we construct a feature denoting the slope of the order book volume. For example, cross-level ask/bid difference between level 2 and level 1 is defined as:

$$\begin{aligned}
 \text{Ask size difference } (t) &= 2^{\text{nd}} \text{ best ask volume} - 1^{\text{st}} \text{ best ask volume} \\
 \text{Bid size difference } (t) &= 2^{\text{nd}} \text{ best bid volume} - 1^{\text{st}} \text{ best bid volume} \\
 \text{Delta cross-level ask/bid difference for level 2 and level 1} \\
 &= (\text{ask size difference } (t) \\
 &\quad - \text{bid size difference } (t)) - (\text{ask size difference } (t-1) \\
 &\quad - \text{bid size difference } (t-1))
 \end{aligned}$$

We also employ the same method to construct the delta cross-level ask/bid difference between level 3 and level 2, level 4 and level 3, and level 5 and level 4. It turns out that the the feature between level 3 and level 2 has the most significant predictive power in the test set.

An important assumption for this feature is that the slope of order book volume is symmetric in ask and bid side, which is reasonable. That means, if the whole market has no consensus on the price, some investors expect a higher price and sell more at best 2 bid price. This will lead to a future price increment. Based on our research results, the delta cross-level ask/bid difference has a positive relationship with VWAP, and it matches our intuition above.

3.2.3 Imbalance Change Rate

For five levels separately, at each quote we look back into a time frame, which is a certain number of quotes in the orderbook, and we measure the change of imbalance in this time frame. As such, imbalance change rate can be computed using the shaded area under the imbalance curve below:

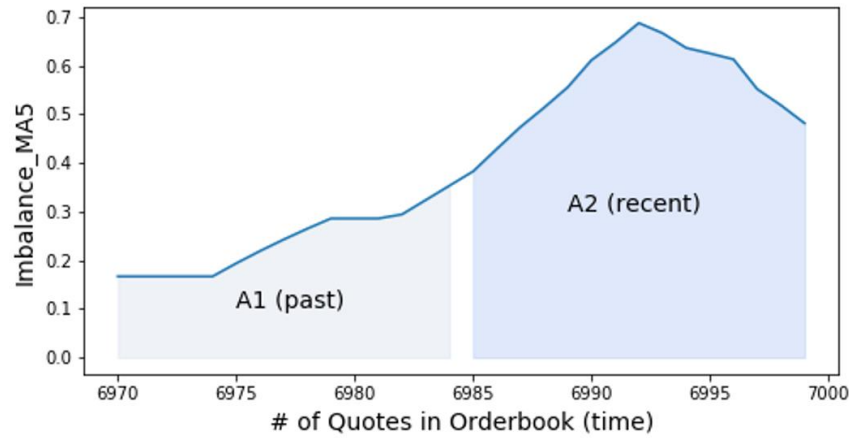


Figure 1: Imbalance Change Rate w.r.t. Orderbook Quotes

$$\text{Imbalance Change Rate} = A2 (\text{recent}) / A1 (\text{past}) - 1$$

The logic behind this category of features is two-fold. Imbalance reflects the relationship of supply and demand in an orderbook. The imbalance change rate can describe the variation of such a relationship, so it may add more predictive power to the dynamics of an orderbook. Compared with static imbalance levels, incorporating a time frame helps smoothen the data and reduce noise.

The length of the time frame (i.e. the number of quotes) is a parameter to be determined in this feature. We used an Information Gain measure to determine the proper time lag.

$$IG (\text{feature}) = H (\text{target}) - H (\text{target} | \text{feature})$$

$H (\text{target})$ is the entropy of the target random variable, which generally refers to disorder or uncertainty in stochastic sources of data. $H (\text{target} | \text{feature})$ is the conditional entropy of the target given the feature. Therefore, IG measures the reduction

of uncertainty. A larger value of IG can indicate stronger relationships. By this method, we found the optimal time lags are between 10 and 50 quotes.

In addition, we placed a strengthening condition to these imbalance change rates by combining them with instantaneous imbalance levels. Specifically, based on the above computations, we also consider the imbalance level at the end of each time frame. We record the increase of imbalance only if the eventual imbalance is toward the bid side, and conversely, record the decrease of imbalance only if the eventual imbalance is toward the ask side. In this way, we only keep the observations where imbalance is really strengthened, but discard the observations where minor changes happen.

3.2.4 Arrival/Deletion Intensity

Apart from time sensitive features constructed from order book prices and quantities, we also looked at trading records to find dynamics in history. Thus, we constructed the intensity features which are the arrival and deletion rates of ask and bid orders times their quantities, respectively. We investigated it for a short period ($\Delta t = 30$ seconds) and a long period ($\Delta T = 300$ seconds).

Intensified arrival/deletion rates of orders may lead to faster changes in the order book. Thus, it can be an increasing/decreasing driving force of prices. Deletion rates can be an opposite driver. The graph below demonstrates the potential relationship between the ask order adding rate (red area) and the level 1 ask price (blue line). We see that they largely move in the same directions throughout the time buckets (horizontal axis):

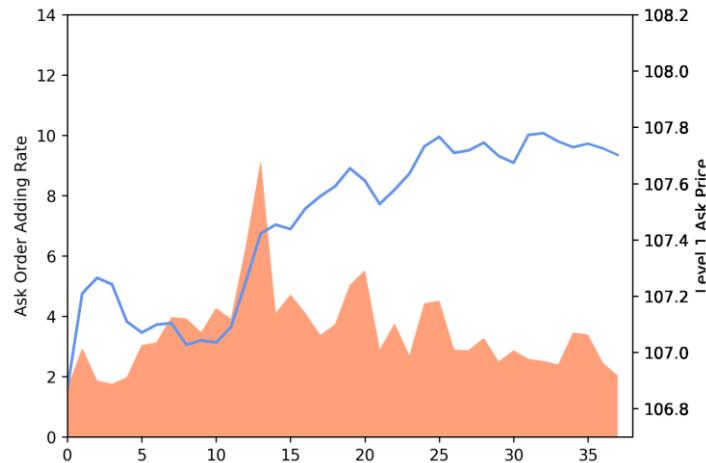


Figure 2: Ask Order Adding Rate and Level 1 Ask Price on Time Buckets

By comparing the short period and long period intensities, we constructed Relative Intensity dummy variables indicating whether the trading type has intensified in the most recent past.

3.3 Cointegration

3.3.1 Definition

For this part, we will generate our forward VWAP movement signal from cointegration. Cointegration is the long-term equilibrium between the response and features. For instance, if there is cointegration between security A and security B and the price of security A goes up, then the price of security B will also change accordingly. Thus, we can model the expected price of security B and derive the deviation of realized price from the expected price. The deviation can be used as the cointegration feature to predict the forward price movement. The reason is, if the price of security B has a mean-reversion pattern, then the forward price of security will converge to the expected price. In other words, the forward price movement will have negative correlation with the deviation. On the other hand, if the price of security B has a momentum pattern, then the forward price will deviate from the expected price even larger and the forward price movement may have positive correlation with the deviation. For the prediction of the forward price of 30-year treasury, the first step is to find securities that may share similar trends with 30-year

treasury. From the future price and spot price of 30-year security (Figure 3), we observed similar trends of price movement. We also observed similar trends of price movement in treasuries with different maturities in Figure 4. Thus, we conducted research on two kinds of cointegration. We will demonstrate our results for future-spot cointegration and cash cointegration respectively.

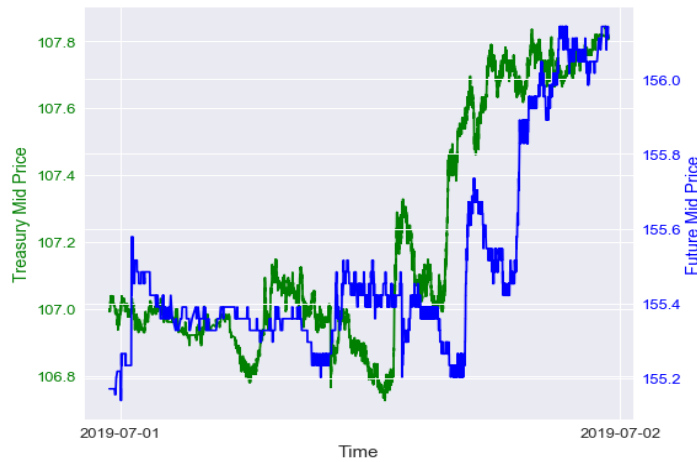


Figure 3: Future Price and Spot Price of 30-year Treasury

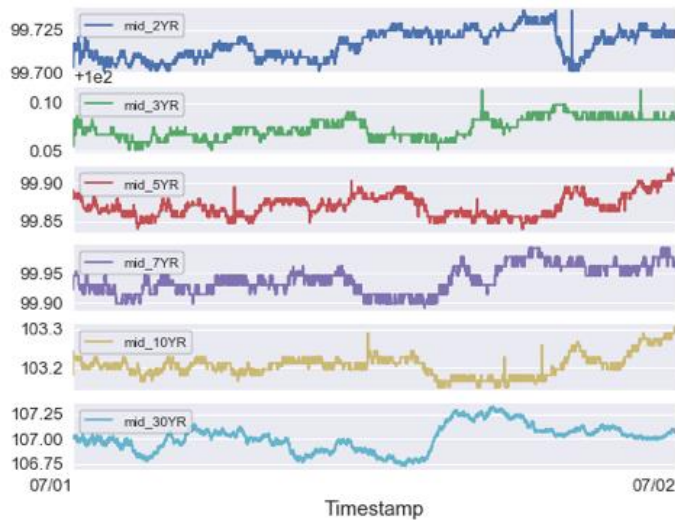


Figure 4: Spot Prices of Treasuries with Different Maturities

3.3.2 Methodology

In cointegration part, our response is still the VWAP change with respect to the mid-price of 30-year security. To our cointegration feature, we first modeled the mid-price of 30-year treasury using the mid-price of 30-year treasury future or other treasuries and then got the residual from the established model. For some of our approaches, we further applied Kalman filter to derived residual and updated the estimates dynamically. We explored 4 approaches to estimate the cointegration feature.

3.3.2.1 Linear Regression + Kalman filter

In the first approach, we used linear regression to predict the mid-price of 30-year treasury with our features and derived residuals accordingly. The linear regression was only applied to the training set once and we assumed the estimated model coefficients would also hold for the test set. We applied unit root test to obtained residual series and found the residual series were stationary process. After that, we applied Kalman filter to update our estimation for the cointegration residuals.

The intuition behind Kalman filter is to update the estimation of the distribution of a certain feature with observed information from prior states. In our case, we obtained the prior distribution of the cointegration residual from our training set. When we processed the residual tick by tick, we observed new residuals and the distribution of those residuals. We combined the observed distribution and prior distribution of the current residual to derive the posterior distribution of residuals. The posterior mean was updated as the estimation for the current residual and the posterior distribution of the current tick was updated as the prior distribution of the next residual. In this way, we predicted our cointegration residuals dynamically tick by tick.

3.3.2.2 Rolling Window Linear Regression + Kalman filter

Considering the linear model coefficients may not hold for the whole dataset, we used rolling window approach to update our model coefficients dynamically. We kept the window length to be 100 ticks. Since rolling-window linear regression only utilize the latest tick data within the training window to estimate the model coefficients, it can better

predict the expected price of 30-year security if the cointegration changes rapidly. We also applied Kalman filter to further update our estimation of cointegration residuals.

3.3.2.3 Online Regression

Online regression is essentially a weighted least square linear regression in a rolling window manner. estimation with an exponential decay rate for the sample importance of historical data. For a linear model $Y=X\beta+\epsilon$, weighted least square approach gives a closed form estimation as:

$$\hat{\beta} = (X^T W X)^{-1} X^T W Y$$

Where $W = \text{diag}(w_i)$, $w_i = \alpha^{n-i}$ is exponentially decayed for older data and $\alpha \in (0,1)$ is the weight decay parameter.

A step by step update for online regression can be summarized as:

$$\begin{aligned} V_t &= \alpha V_{t-1} + (1 - \alpha) X^T Y \\ M_t &= \alpha M_{t-1} + (1 - \alpha) X^T X \end{aligned}$$

Where the estimated model coefficients at t is given by:

$$\tilde{\beta} = M_t^{-1} V_t$$

In our case, we applied online regression tick by tick and simply used the residuals obtained from the regression model as our estimation for cointegration feature.

3.3.2.4 Rolling Window Gaussian Process Regression

Since the previous three methods only models the linear relationship between different securities, we introduced Gaussian Process Regression to model more complex and non-linear relationship between different securities.

Gaussian Process Regression is a universal approximator like neural network, which can approximate any continuous function. It's a non-parametric Bayesian approach towards the regression problem, while both the prior and posterior distributions are multi-variate

normal distributions. The prediction of gaussian process regression gives the estimated distribution of test data with mean value as point estimate.

The regression model is given as follow:

$$y = f(x) + \epsilon$$

Where $f(x) \sim GP(m(x), K(x, x))$ and $GP(m(x), K(x, x))$ represents Gaussian Process prior. The model assumes the training observations and expected test data have multivariate normal distribution as:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} K(X, X) + \sigma_n^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix} \right).$$

Thus, we can derive the predictive distribution of test data as:

$$\begin{aligned} f_* | X_*, X, Y &\sim N(E[f_* | X_*, X, Y], \text{var}[f_* | X_*, X, Y]) \\ E[f_* | X_*, X, Y] &= \mu_{X_*} + K(X_*, X)[K(X, X) + \sigma^2 I]^{-1} Y \\ \text{var}[f_* | X_*, X, Y] &= K(X_*, X) - K(X_*, X)[K(X, X) + \sigma^2 I]^{-1} K(X, X) \end{aligned}$$

We applied gaussian process regression to predict mid-price of 30-year Treasury and derived the residuals directly as our estimation of cointegration feature.

3.3.3 Cash Cointegration

We selected 2-year, 3-year, 5-year, 7-year, 10-year treasury as our features to predict the price of 30-year treasury. The VWAP6 change with respect to 4 cointegration features is displayed in Figure 5 to Figure 8.

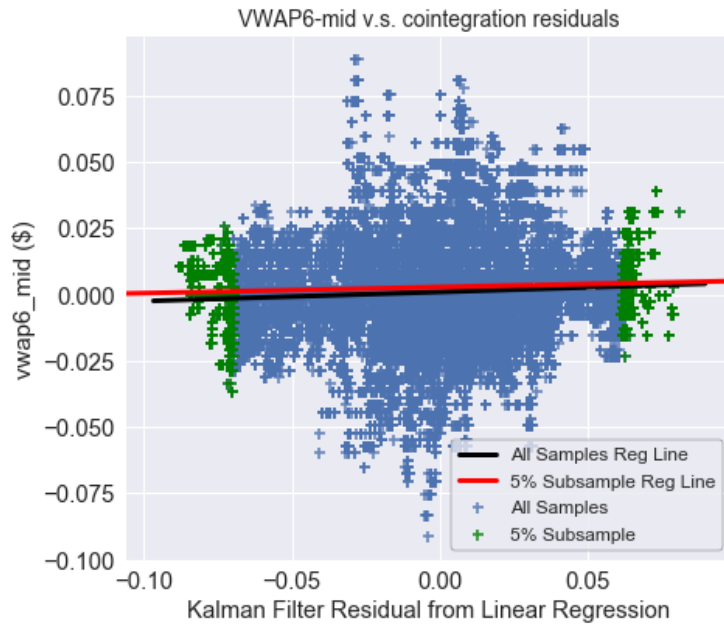


Figure 5: Response w.r.t Linear Regression Residuals

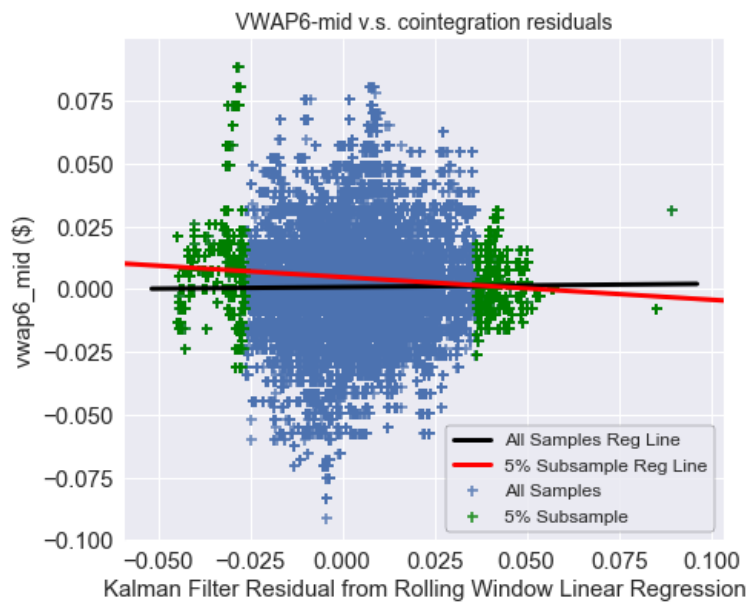


Figure 6: Response w.r.t Rolling Window Linear Regression Residuals

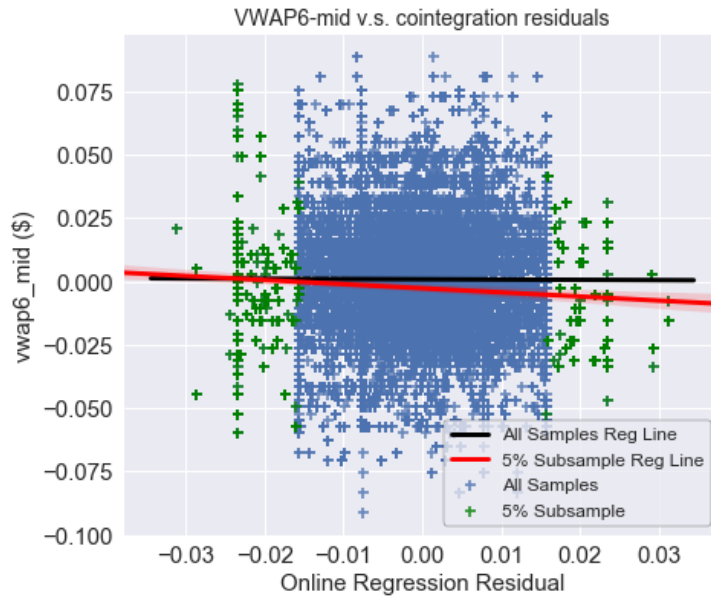


Figure 7: Response w.r.t Online Regression Residuals

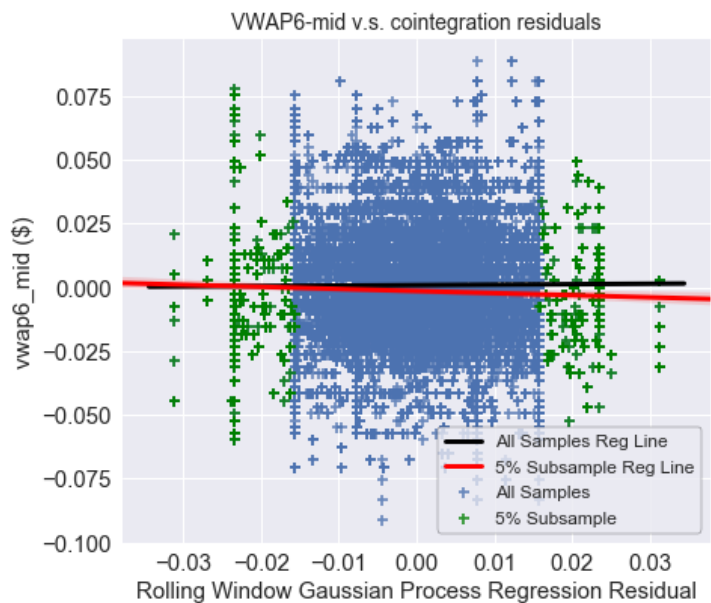


Figure 8. Response w.r.t Gaussian Process Regression Residuals

We found cointegration feature from linear regression approach showed slight momentum pattern of mid-price of 30-year treasury. However, the cointegration features from rolling window linear regression, online regression and rolling window gaussian process regression implied mean-reversion pattern. For those features, though the slopes

of regression lines of all samples were approximately 0, the regression lines for the outliers (smaller than 2.5% quantile or larger than 97.5% quantile) had negative slopes, indicating a mean-reversion pattern. Thus, we incorporated those cointegration features into our trading strategies to verify whether we could utilize such patterns to generate efficient execution strategies.

However, we also found that mean-reversion pattern was more apparently identified in 5-year and 10-year treasury. The regression lines for both all samples and outliers demonstrated negative slopes. The patterns can be seen in Figure 9 and Figure 10. Thus, we used 5-year and 10-year treasury as a benchmark to compare the strategy performance with that of 30-year treasury.

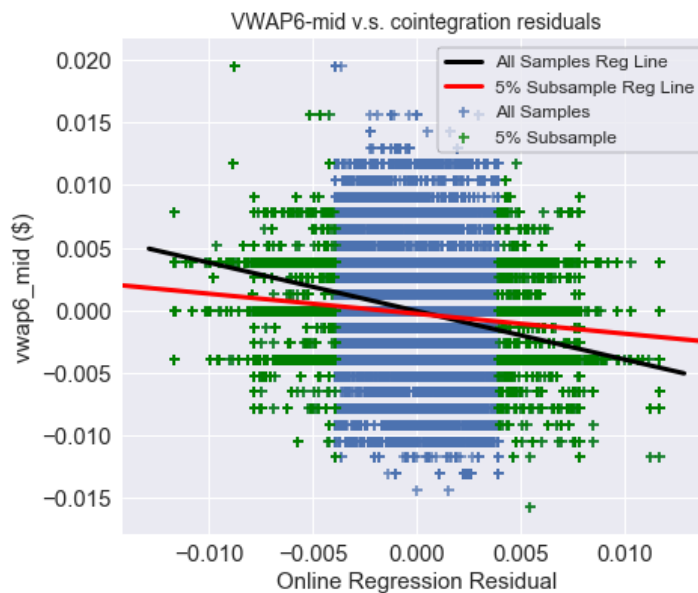


Figure 9: Response w.r.t Online Regression Residuals for 5-year Treasury

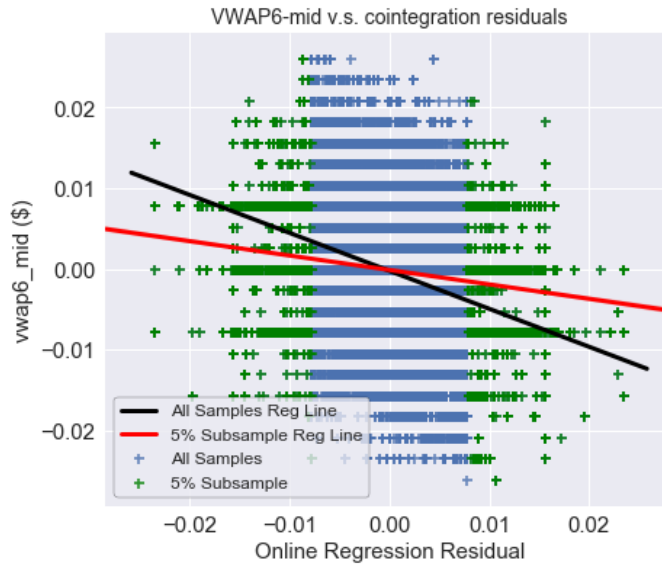


Figure 10: Response w.r.t Online Regression Residuals for 10-year Treasury

3.3.4 Spot Future Cointegration

For the future spot cointegration, we selected the mid-price of the corresponding bond future (ZBU9: MBO) as our features to predict the price of 30-year treasury. Again, we used four different dynamic models in our cointegration feature construction and the VWAP6 change with respect these four cointegration features are displayed in Figure 11 to Figure 14.

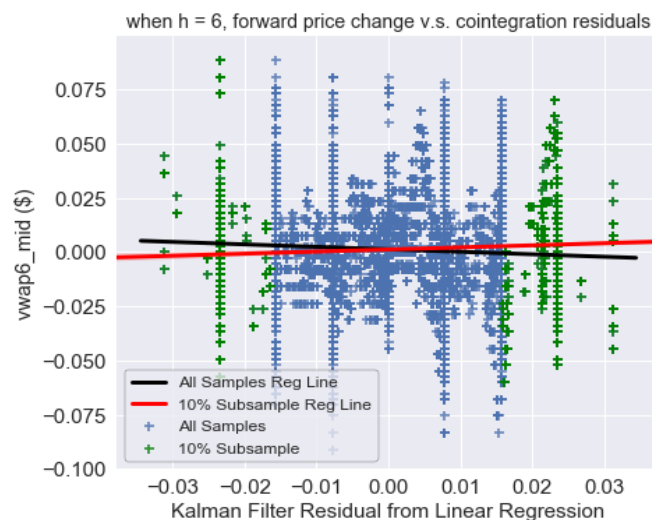


Figure 11: Response w.r.t Window Linear Regression Residuals

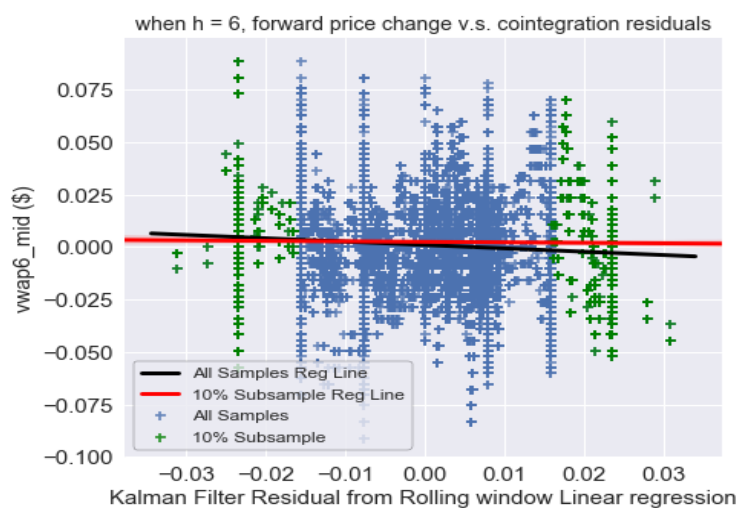


Figure 12: Response w.r.t Rolling Window Linear Regression Residuals

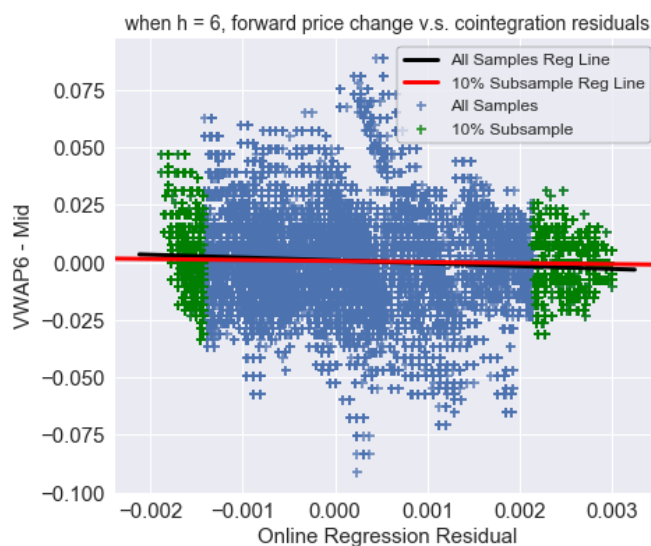


Figure 13: Response w.r.t Online Regression Residuals

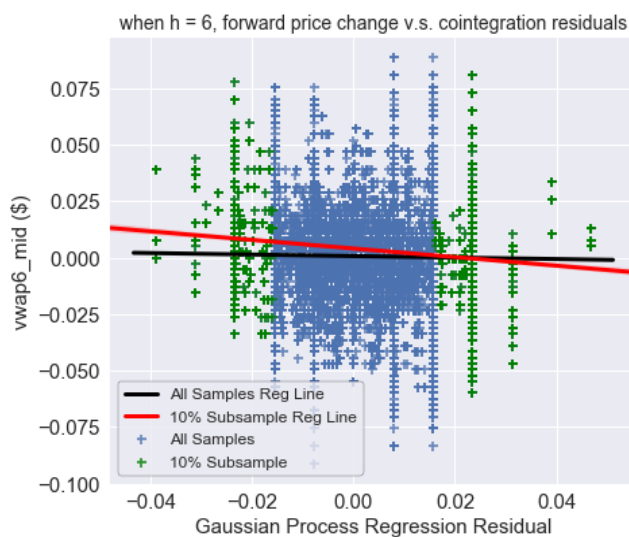


Figure 14: Response Gaussian Process Regression Residuals

As can be seen from the plots above, firstly all the constructed cointegration features based on bond future is closely distributed around zero, suggests there is a solid cointegration relationship between the mid-price of 30-year treasuries and its corresponding future. Also, similarly to the cash cointegration, although there seems to be a slight momentum patten in the basic linear model, the cointegration features from

other more flexible models implied mean-reversion pattern with noticeable negative slopes of the regression lines for the outliers and even some significant negative slopes for the regression lines for all data points (rolling window regression). Thus, we also incorporated those cointegration features into our trading strategies to verify whether we could utilize such patterns to generate efficient execution strategies.

In addition, we also constructed the same cointegration features and draw the residual plots for our 5-year and 10-year benchmark (Figure 15, Figure 16). Same as the cash cointegration, we found that mean-reversion pattern was more apparently identified in 5-year and 10-year treasury.

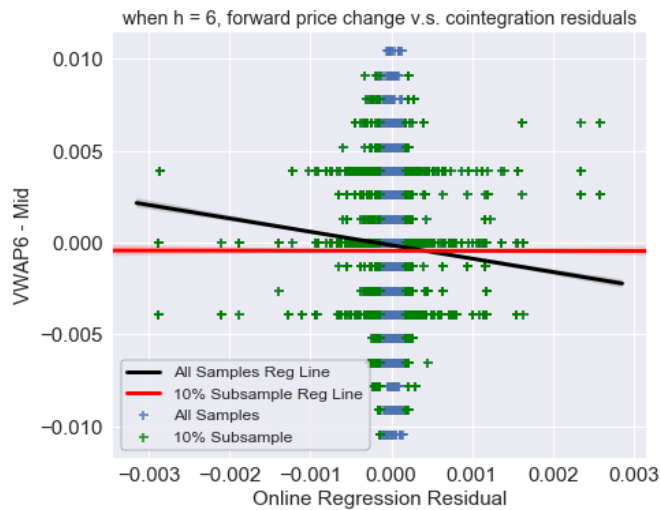


Figure 15: Response w.r.t Online Regression Residuals for 5-year Treasury

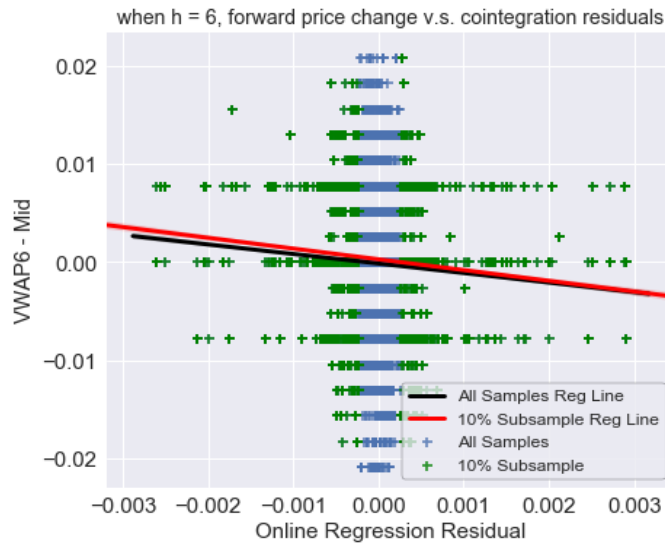


Figure 16: Response w.r.t Online Regression Residuals for 10-year Treasury

3.3.5 Cointegration Summary

- There are strong cointegration relationships between treasuries of different duration as well as between 30-year treasury and future.
- For the Cash Cointegration, while the residuals for the linear regression shows slight momentum, the residuals of all other models show a Mean-Reversion pattern and there are more apparent Mean-reversion patterns appearing in the 5 Year and 10 Year securities which we use as our benchmarks.
- For the Spot Future Cointegration, we found similar results, while the residuals for the linear regression shows no significant pattern, all other models show a Mean-reversion pattern, which indicates the signal we constructed out of cointegration is closely related to the VWAP movement.

4. Modelling

4.1 Online Regression

Our final trading strategy was based on the fair-price prediction for each quote, so it was critical to develop an appropriate model to capture the relationship between our response and the above proposed features. Here we took the 6 million VWAP as our response. Our goal became to model the function

$$Y = VWAP_6 - Mid = f(X_1, X_2, \dots, X_d)$$

Where $X_1, X_2, \dots, X_d$ are any potential combination of features. We also took the difference of VWAP and the mid-price as our response in practice to ensure stationarity of the response.

We used online regression model for this problem, which had also been introduced in cointegration modelling mentioned previously,

$$Y = X\beta + \epsilon$$

the coefficients were estimated by

$$\hat{\beta} = (X^T W X)^{-1} X^T W Y$$

Where W is the diagonal weight decay matrix. The online regression provided several advantages in this case: first, it was easy to interpret and help us to understand the relationship between the response and individual features; second, it was efficient in computation; third, it was a dynamic method that assigned more weights to later points but less weights for older points, so we were able to constantly adjust our coefficients and keep up with latest trend.

4.2 Batch Update

A trick that we used here for online regression was that we were not updating our coefficients whenever a new observation of quote came, but updating them after a batch of observations. We applied this trick for two main reasons: first, the response we

constructed contained future information, so to avoid information leakage, we were not allowed to train our model and update the coefficients using these observations; second, due to the fact that the trades came less frequent than the quotes, the later quote was not necessarily more informative than older quote since there were no actual trades happening between them. Simply setting weight decay for every quote excessively overlooked historical information and introduced noise.

This methodology worked in the following steps:

- We first ran OLS in the initial training dataset to obtain the coefficients
- Then we went through every quote in the out-of-sample dataset. For each quote, we predicted VWAP using the latest updated coefficients
- Lastly, we determined whether we could update our coefficients. We used a queue to record past quotes, and we added the latest quote in the queue. We then checked if we observed response of any quotes in the queue. If not, we were not allowed to update our model by any of these quotes. Otherwise, we must have obtained the responses of some quotes in the queue, so we updated our model based on this batch of quotes and then removed these quotes from the queue

To be explicit for the online regression, the update step was

$$\begin{aligned} V_t &= \alpha V_{t-1} + (1 - \alpha) X_{batch}^T Y_{batch} \\ M_t &= \alpha M_{t-1} + (1 - \alpha) X_{batch}^T X_{batch} \end{aligned}$$

So the updated coefficients became

$$\tilde{\beta} = M_t^{-1} V_t$$

5. High Frequency Trading Strategy

5.1 Benchmark

Firstly, we constructed a TWAP trading strategy as the benchmark. Every 5 minutes, we buy and sell one million of the last quote at the best bid and ask price, and calculate the average transaction cost of the difference between the buying and selling price.

Also, we consider one tick-size as another benchmark, which is 0.015625 based on the 30-year treasury bond data.

5.2 Smart Algorithm

We constructed two smart algorithms. One is a tick-by-tick strategy, and the second is a 30-second sampling trading strategy.

5.2.1 Tick-by-tick strategy

For each quote we calculated the features' value and input into the online regression to predict the fair price, $\hat{P}$. Then, we took the absolute value of $\hat{P}$ and calculated 10% quantile separately and set them as buy or sell thresholds. The trading signal is designed as follows:

$$\text{Buy signals: } |P_{ask} - \hat{P}| < \text{Threshold}_{buy}.$$

$$\text{Sell signals: } |\hat{P} - P_{bid}| < \text{Threshold}_{sell}.$$

Every 5 minutes, we buy and sell the first quote that trigger the buy/sell signals. If no quote has been triggered, we buy and sell the last quote of 5 minutes which is the same as the TWAP algorithm.

5.2.2 30-second strategy

Every 30 seconds, we subsample the data by taking the last quote of 30 seconds.

Therefore, we have 10 observations every 5 minutes. The buy and sell signals are designed the same as the tick-by-tick strategy.

Every 5 minutes, we buy and sell the first observations that trigger the buy/sell signals. If no quote has been triggered, we buy and sell the last observation of 5 minutes which is the same as the TWAP algorithm.

5.3 Backtesting Result

The TWAP benchmark result is 0.01899.

5.3.1 Single Feature Trading Strategy

We input each feature into the online regression model and predict the fair-price. Then we calculated the average transaction cost, the result is shown below:

Table 1: Single Feature Trading Strategy Results

Group of Features	Feature	Tick-by-tick Strategy		30-Second Sampling Strategy	
		Average Transaction Cost	Proportion of Tick Size	Average Transaction Cost	Proportion of Tick Size
Time insensitive	Imbalance Lvl 1 Cubed	0.0044	0.2816	0.0087	0.5568
	Imbalance Lvl 1	0.0073	0.4672	0.0089	0.5696
	Sum_qdiff	0.0091	0.5824	0.0113	0.7232
Time Sensitive	ImbChg Lvl 1 Lag 10 Strengthened	0.0079	0.5056	0.0166	1.0624
	ImbChg Lvl 2 Lag 50	0.0105	0.6720	0.0146	0.9344
	Dummy Mid-price Change	0.0133	0.8481	0.0170	1.0886
	Relative Intensity – Ask Arrival	0.0158	1.0127	0.0148	0.9494
Cointegration	Futures – Rolling Window + Kalman Filter	0.0093	0.5952	0.0133	0.8512
	Cash - Rolling Window + Kalman Filter	0.0093	0.5952	0.0188	1.2032

From the result we can observe that imbalance level one cubed, original imbalance one and sum of quantity difference cross levels from time-insensitive groups can beat one tick-size. The change of Imbalance level one with lag 10, dummy mid-price change and relative intensity ask-arrival from time-sensitive features group can beat one tick-size.

The futures cointegration features with rolling window and Kalman filter, and cash cointegration with rolling window and Kalman filter from cointegration features group can beat one tick-size. Also, all of the features above can beat TWAP algorithm.

Therefore, we can conclude that the features we create from different perspectives can be effective for the prediction of fair price as well as the reduction of transaction cost.

5.3.2 Multiple Feature Trading Strategy

After getting the performance of single feature trading strategy, we consider whether combining features with good performance can generate an even better result. When we look for combinations of features, we first compute the pairwise correlations and Variance Inflation Factors (VIF) of all features in order to avoid high collinearity. We select low-correlated, top-performed features from each group, e.g., time sensitive, time insensitive and cointegration features, and construct different combinations of features to input into the online regression model. Then we can implement the same trading strategy based on our fair price prediction. The results are shown below:

Table 2: Multiple Feature Trading Strategy Results

Features Combination	Tick-by-tick Strategy		30-Second Sampling Strategy	
	Average Transaction Cost	Proportion of Tick Size	Average Transaction Cost	Proportion of Tick Size
Imbalance 1 Cubed + Imbalance Change Strengthened + Futures Cointegration	0.0008	0.0512	0.0026	0.1664

Imbalance 1 Cubed + Futures Cointegration	0.0016	0.1024	0.0044	0.2816
Imbalance 1 Cubed + Imbalance Change Rate + Futures Cointegration	0.0022	0.1408	0.0036	0.2304
Imbalance 1 Cubed + Imbalance Change Rate + Cash Cointegration + Futures Cointegration	0.0026	0.1664	0.0028	0.1792

From the result we can observe that the combination of imbalance level one cubed, imbalance change strengthened and futures cointegration features can help to reduce the transaction cost by 95% compared with one tick-size. Also, this combination outperforms every single features trading strategy. Therefore, we can conclude that combining the best feature within each group can extract information from different perspectives and massively reduce the transaction cost.

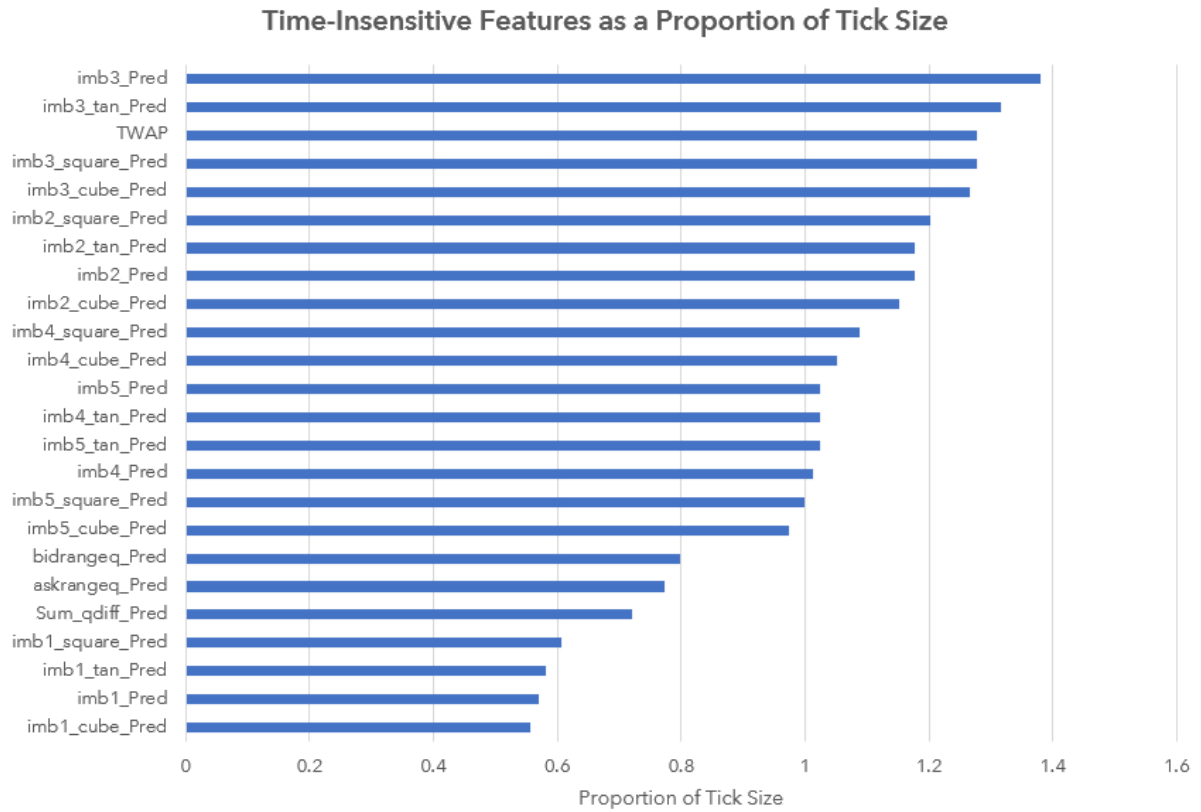
6. Conclusion

We found that the cubic transformation for imbalance level one can extract more information to make predictions for the fair-price as compared to the original value of imbalance level one for 30 year US Treasury Bonds, and the transaction cost can be reduced by 71% compared with one tick-size. Moreover, we combined the features with the best performance from different groups of features, e.g., time-sensitive features, time-sensitive features and cointegration features, to construct a multiple features trading strategy. The result shows that 95% of transaction costs can be reduced compared to one tick-size, and 87% of transaction cost can be reduced compared to just using imbalance level one. Therefore, we believe that the new features we construct can extract more information from order-book and help to massively reduce the transaction cost.

7. Appendix

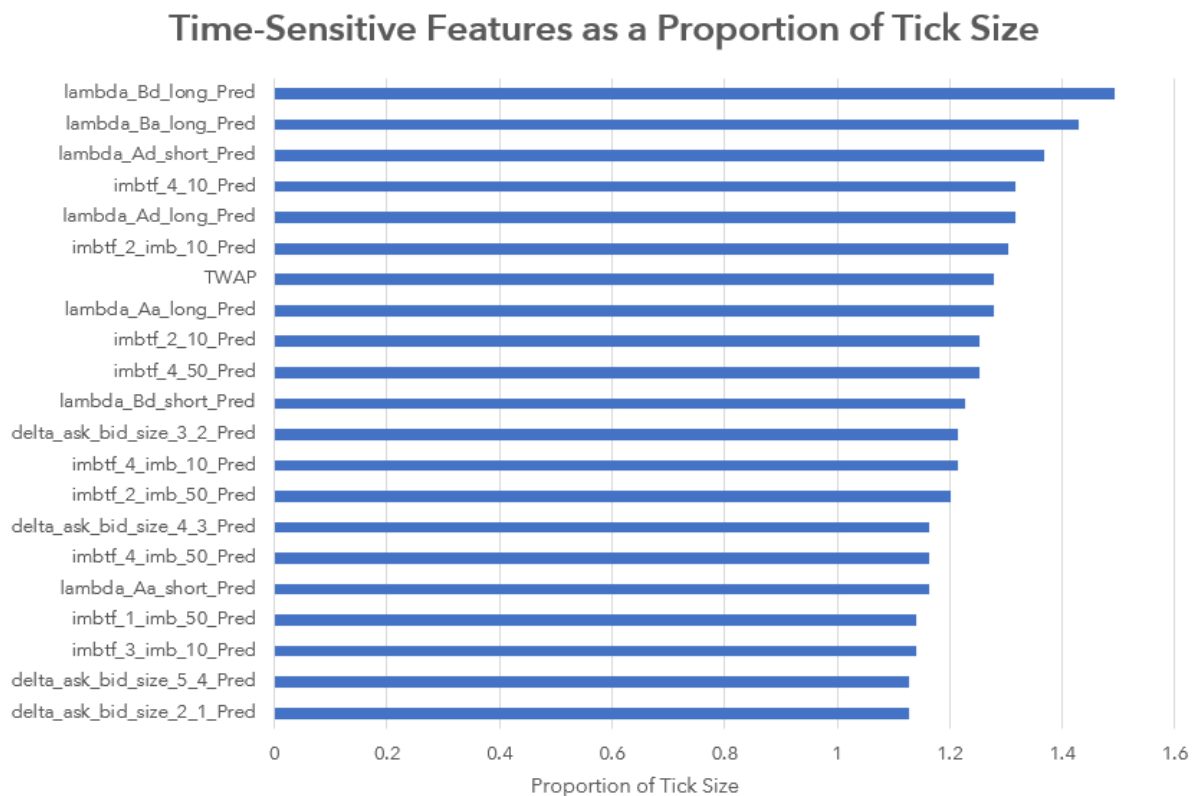
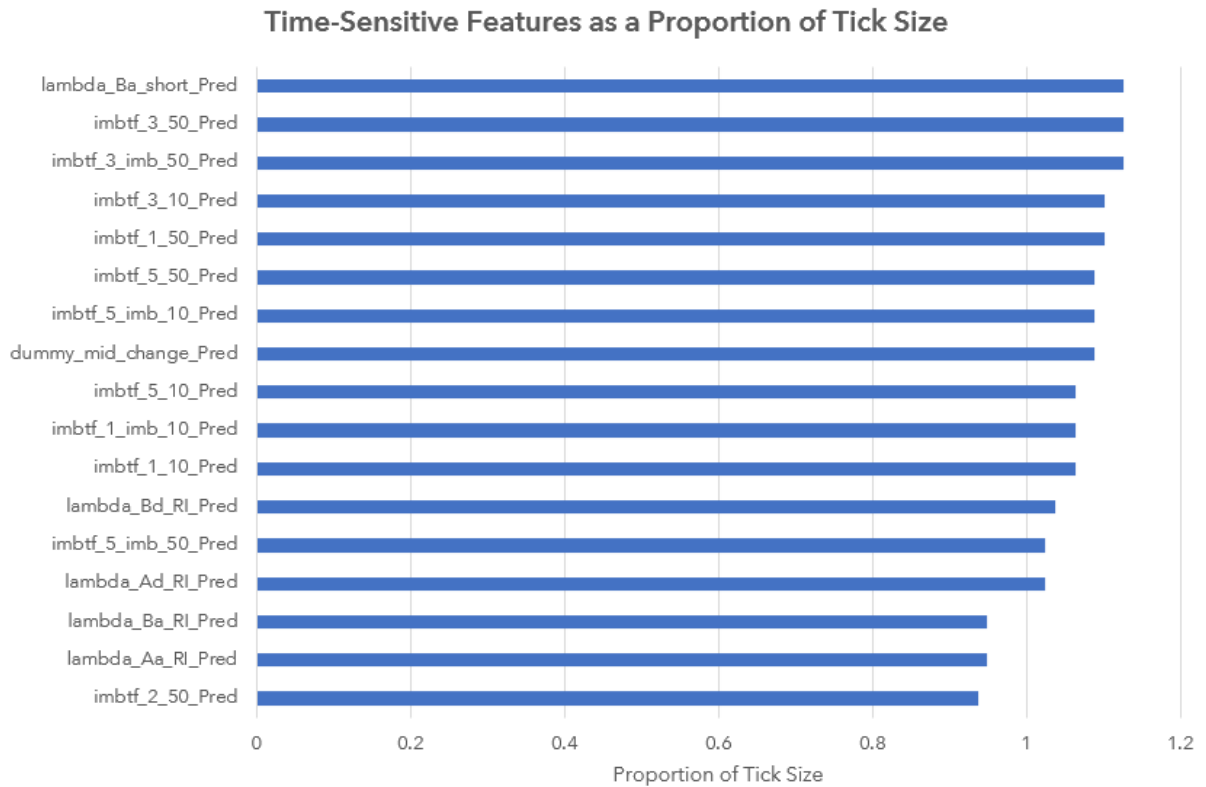
7.1 Appendix 1: Time-Insensitive Features Results of 30-second Sampling Strategy in Test Set

One tick size = 0.015625



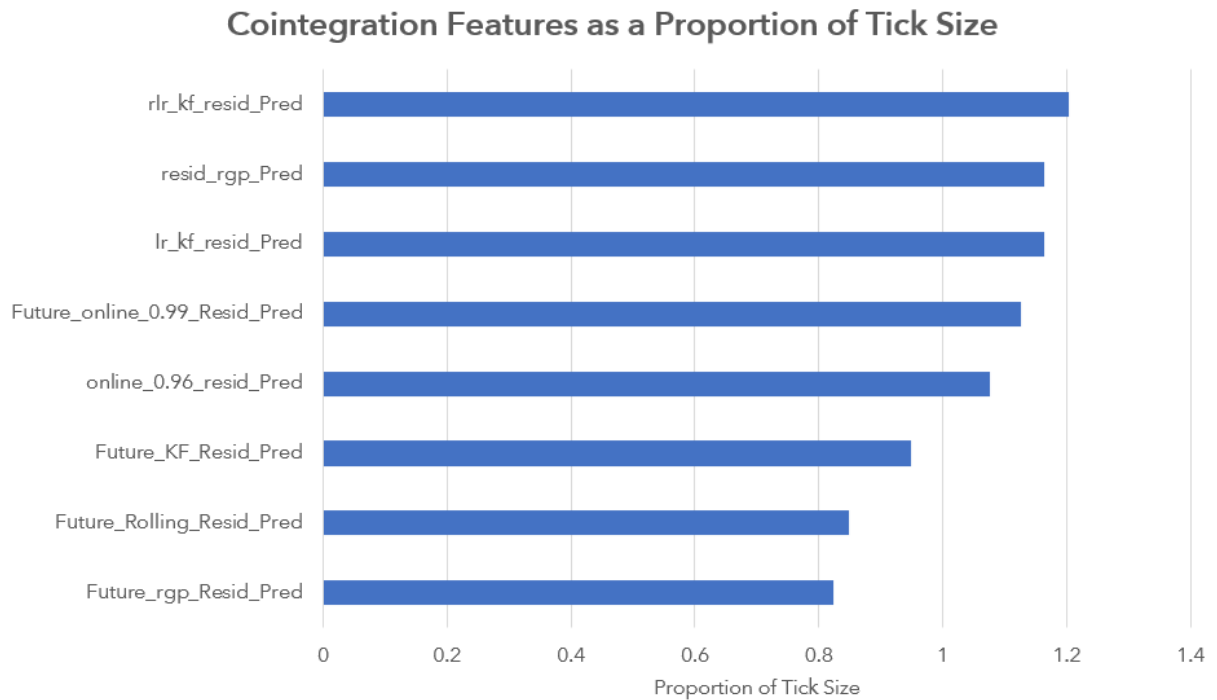
7.2 Appendix 2: Time-Sensitive Features Results of 30-second Sampling Strategy in Test Set

One tick size = 0.015625



7.3 Appendix 3: Cointegration Features Results of 30-second Sampling Strategy in Test Set

One tick size = 0.015625



7.4 Appendix 4: Multi-Features Results of 30-second Sampling Strategy in Test Set

Benchmark (imbalance level 1) = 0.00633

